

Java Advanced Schulung 2026

Lea Föcke & Phil Steinhorst



Lambda-Ausdrücke

- Lambda-Ausdrücke sind anonyme (= namenlose, ungebundene) Funktionen.
- **Funktion:** Oberbegriff für Methode, aber nicht notwendig zu einer Klasse gehörend.
- Allgemeine Syntax: `(param1, param2, ..., paramN) -> {Code-Block}`
- Bei nur einem Parameter können die Klammern fortgelassen werden:
`param -> {Code-Block}`
- Ebenso bei nur einer Anweisung im Code-Block (analog `if`, `while`, etc.)
`(param1, param2, ... paramN) -> Anweisung`

Verwendung von Lambda-Ausdrücken

- Häufige Verwendung für "kurze" Berechnungsvorschriften.

Als Methode:

```
public double mittelwert (double x, double y) {  
    return (x + y) / 2;  
}
```

Als Lambda-Ausdruck:

```
(x, y) -> (x + y) / 2;
```

- Lambda-Ausdrücke können – wie andere Werte auch – als Parameter und Rückgabe verwendet sowie Bezeichnern zugewiesen werden:

```
List<Integer> ls = List.of(4, 8, 15);  
ls.forEach( x -> System.out.println(x * 2) );  
// Ausgabe:  
8  
16  
30
```

- Welchen Datentyp hat der Lambda-Ausdruck?

Datentypen von Lambda-Ausdrücken

forEach

```
public void forEach(Consumer<? super E> action)
```

- Die Methode `forEach` von `List<E>` verlangt ein `Consumer<? super E>`.

```
@FunctionalInterface  
public interface Consumer<T>
```

Represents an operation that accepts a single input argument and returns no result. Unlike most other functional interfaces, `Consumer` is expected to operate via side-effects.

- `Consumer<T>` ist ein **functional interface** für Lambda-Ausdrücke mit einem Parameter von Typ `T` und ohne Ergebnis (`void`).

Speichern von Lambda-Ausdrücken

- Wir können `Consumer` nutzen, um einen Lambda-Ausdruck zu speichern...

```
Consumer<Integer> fun = x -> System.out.println(2 * x);  
List<Integer> ls = List.of(4, 8, 15);  
ls.forEach(fun);
```

- ... oder um ihn als Resultat einer Methode zurückzugeben:

```
public Consumer<Integer> nMalAusgeben(int n) {  
    return x -> {  
        for (int i = 0; i < n; i++)  
            System.out.println(x);  
    };  
}
```

```
public Consumer<Integer> nMalAusgeben(int n) {  
    return x -> {  
        for (int i = 0; i < n; i++)  
            System.out.println(x);  
    };  
}
```

```
var dreiMalAusgeben = nMalAusgeben(3);  
dreiMalAusgeben.accept(5);  
// Ausgabe:  
5  
5  
5
```

Functional Interfaces

- `@FunctionalInterface` zeigt an, dass das Interface **genau** eine abstrakte Methode (= nicht `@Override` und ohne `default`-Implementierung) definiert.
- Diese legt fest, welche Lambda-Ausdrücke verwendet werden können

```
@FunctionalInterface
public interface Consumer<T> {
    // abstrakt, da ohne default-Implementierung
    // Eingabe: Objekt vom Typ T; Rückgabe: nichts (void)
    void accept(T t);

    // nicht abstrakt, da default-Implementierung vorhanden
    default Consumer<T> andThen(Consumer<? super T> after) {
        /* ... */
    }
}
```

- Definiert man in einem functional interface mehr als eine (oder gar keine) abstrakte Methode, führt dies zu einem Compile-Fehler.

```
@FunctionalInterface
public interface NichtFunktional {
    int eineMethode(int a, int b);
    double andereMethode(double a, double b);
}
```

```
java: Unexpected @FunctionalInterface annotation
NichtFunktional is not a functional interface
multiple non-overriding abstract methods found in interface NichtFunktional
```

- Ganz allgemein kann man **jedes** Interface, das genau eine abstrakte Methode definiert, durch einen Lambda-Ausdruck instanziiieren:

```
// mit oder ohne @FunctionalInterface-Annotation
public interface MeinInterface{
    int machWas(int x);
}
```

```
MeinInterface fun = x -> 2*x;
fun.machWas(5); // Rückgabe: 10
```

- Die Annotation verhindert nur, dass man "versehentlich" eine weitere abstrakte Methode definiert...
- ... und weist Entwickler*innen darauf hin, dass man bequem einen Lambda-Ausdruck verwenden kann.

Was passiert beim Instanziieren?

```
public interface MeinInterface{  
    int machWas(int x);  
}  
  
MeinInterface fun = x -> 2*x;
```

- Technisch wird zur Laufzeit eine anonyme Klasse erzeugt, die das Interface implementiert und dafür den Rumpf des Lambda-Ausdrucks verwendet.

```
class OhneName implements MeinInterface{  
    int machWas(int x) {  
        return 2*x;  
    }  
}  
  
OhneName fun = new OhneName();
```

- In `java.util.function` sind Functional Interfaces für übliche Kombinationen von Eingaben und Rückgaben vordefiniert:
 - `Function<T,R>` : ein Parameter vom Typ `T` ; Rückgabe vom Typ `R` .
 - `BiFunction<T,U,R>` : ein Parameter vom Typ `T` , einer vom Typ `U` ; Rückgabe vom Typ `R` .
 - `BinaryOperator<T>` : zwei Parameter und Rückgabe vom Typ `T` .
 - ... (siehe [Doku zu java.util.function](#))
- Da Java nicht erraten kann, welches Interface man implementieren möchte: Lambda-Ausdrücke müssen immer **explizit** typisiert werden.

```
var mittel = (a, b) -> (a+b) / 2;
```

Cannot infer type: lambda expression requires an explicit target type

Nicht vordefinierte Functional Interfaces

- Für darüber hinausgehende Lambda-Ausdrücke muss man selbst ein Functional Interface definieren.

```
/**
 * Funktionales Interface für Lambda-Ausdrücke mit drei Parametern
 * und Rückgabe vom selben Typ.
 */
@FunctionalInterface
public interface TernaryOperator<T> {
    T apply(T a, T b, T c);
}

TernaryOperator<Double> mittel = (a, b, c) -> (a + b + c) / 3;
mittel.apply(5.0, 13.0, 14.0);
```

Methodenreferenzen

- Methoden können referenziert und genauso verwendet werden wie Lambda-Ausdrücke.
- Syntax für nicht-statische Methoden: `objekt::methode`

```
public class Foo {  
    public void doppelAusgeben(int x) {  
        System.out.println(2 * x);  
    }  
}
```

```
List<Integer> ls = List.of(4, 8, 15);  
Foo f = new Foo();  
ls.forEach( f::doppelAusgeben );
```

- Syntax für statische Methoden: `Klasse::methode`

Mehr Infos zu Lambda-Ausdrücken

- <https://www.baeldung.com/java-8-functional-interfaces>
- <https://www.baeldung.com/java-8-lambda-expressions-tips>